Lab3

Exercise 1: Encapsulation & Access Rights

A junior developer wrote the following class and `main` method for a banking application, but the code **fails to compile**.

Java
```
class BankAccount {
    private String accountNumber;
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
}

public class Ex1_Encapsulation {
    public static void main(String[] args) {
        BankAccount myAccount = new BankAccount();

        // --- ERROR 1 ---
        myAccount.accountNumber = "ACC-100203";

        // --- ERROR 2 ---
        myAccount.balance = 500.0;

        System.out.println("Account Number: " + myAccount.accountNumber);
        System.out.println("Current Balance: $" + myAccount.balance);
    }
}
```

1. Explain why the compiler throws errors on lines marked with `ERROR 1` and `ERROR 2`.
2. Modify the `BankAccount` class and the `main` method to respect encapsulation properly by implementing appropriate **getters and setters** (with validation for deposits).

Exercise 2: Method Access and Invocation

Write a Java program containing a class named `Employee` with the following requirements:

1. Three instance variables: `name` (type `String`), `hourlyRate` (type `double`), and `hoursWorked` (type `int`).
2. A method `setEmployeeDetails(String n, double rate, int hours)` to initialize the attributes.
3. A method `calculateGrossSalary()` that computes and returns the gross salary (`hourlyRate * hoursWorked`).
4. A method `displayPayslip()` that prints the employee's information and **internally invokes** `calculateGrossSalary()` to display the final salary.

5. A `main` method in a separate class (`Ex2_MethodAccess`) to instantiate an `Employee`, set its values, and call `displayPayslip()`.

Expected Execution Result:

Plaintext
```
Employee: Sarah Connor | Rate: $25.0/hr | Hours: 40
Gross Salary: $1000.0
```

Exercise 3: Accessing an Object of Another Class

Design a program that models an online customer and their delivery address using two distinct classes:

1. **`Address` class:** Attributes for `street` and `city` (both `String`), a constructor to initialize them, and a method `showAddress()` to print the address details.
2. **`Customer` class:** Attributes for `customerName`, `phoneNumber`, and an attribute of type `Address` (`address`).
3. Inside `Customer`, write a method `displayCustomerProfile()` that prints the customer's details and **accesses the methods of the embedded `Address` object**.
4. Create a test class (`Ex3_ObjectAccess`) with a `main` method to instantiate an address, pass it to a customer object, and display the complete profile information.

Expected Execution Result:

Plaintext
```
Customer: Med Abdallah
Phone: +216 98 123 456
Delivery Address: 5th Avenue, Tunis
```

Exercise 4: The `this` Reference

In the following constructor, the parameters have the exact same names as the instance variables, causing a bug where the attributes remain uninitialized (`0` and `null`).

Java
```
class Student {
    int id;
    String name;

    // Buggy constructor
    public Student(int id, String name) {
        id = id;
        name = name;
    }

    public void display() {
        System.out.println("Student ID: " + id + ", Name: " + name);
    }
}
```

1. Fix the constructor using the `this` keyword to correctly assign parameter values to the instance variables.
2. Add a method `incrementAndReturnId()` that increases the student's ID by 1 and **returns the current object instance** (`this`).
3. Test your code in a `main` method to verify method chaining (e.g., `s1.incrementAndReturnId().display();`).

Exercise 5: Constructors and Constructor Overloading

Write a Java program defining a `Car` class with attributes `model`, `fuelType`, and `year`. Your implementation must include:

1. A **default constructor** that automatically delegates/chains to the parameterized constructor using `this(...)`, setting default values (e.g., `"Unknown Model"`, `"Petrol"`, `2023`). It should also print `"Default constructor called."`.
2. A **parameterized constructor** that initializes all three attributes.
3. A method `displayCar()` to show the car details.
4. A `main` method that tests both constructors and prints the results.

Expected Execution Result:

Plaintext

```
Default constructor called.
Car Model: Unknown Model | Fuel: Petrol | Year: 2023
Car Model: Tesla Model 3 | Fuel: Electric | Year: 2025
```